



ANKETA ČVUT – ADMIN ROZHRANÍ – PODPORA PRO HLAVNÍHO ADMINISTRÁTORA

Jakub Ďurkovič

Bakalárska práca
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Študijný program: Informatika
Špecializácia: Softwarové inženýrství 2021
Vedúci: Ing. Michal Valenta, Ph.D.
2. októbra 2026



Zadání bakalářské práce

Název:	Anketa ČVUT - Admin rozhraní - podpora pro hlavního administrátora
Student:	Jakub Ďurkovič
Vedoucí:	Ing. Michal Valenta, Ph.D.
Studijní program:	Informatika
Obor / specializace:	Softwarové inženýrství 2021
Katedra:	Katedra softwarového inženýrství
Platnost zadání:	do konce letního semestru 2026/2027

Pokyny pro vypracování

Cílem práce je rozšíření administrátorského rozhraní aplikace Anketa ČVUT. Jedná se o návrh a implementaci několika nových požadavků a opravu několika známých chyb. Jednotlivé úkoly jsou zadány formou issue v repozitáři gitlab. Úkolem studenta je postupně jednotlivé požadavky analyzovat, navrhnout řešení, implementovat a vhodným způsobem zdokumentovat a otestovat.

Jedná se zejména o tyto úkoly:

- analyzovat a popsat správu rolí v databázích Usermap a Anketa,
- zdokumentovat ostatní podobné dávky (jobs), kde jsou formou materializovaných pohledů přebírána data z databáze KOS do databáze Anketa,
- refaktorovat a dokumentovat PL/SQL kód pro přebírání privilegovaných rolí z databáze Usermap,
- navrhnout a rozšířit admin rozhraní o správu privilegovaných rolí,
- opravit a rozšířit evidenci otázek a seznamů otázek o nově implementované otázky typu "1 z N" a "M z N",
- opravit správu administrátorů ankety,
- doimplementovat ovládání ateliérové ankety.

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Jakub Ďurkovič. Všechny práva vyhrazené.

Táto práca vznikla ako školské dielo na FIT ČVUT v Prahe. Práca je chránená medzinárodnými predpismi a zmluvami o autorskom práve a právach súvisiacich s autorským právom. Na jej využitie, s výnimkou bezodplatných zákonných licencií a nad rámec oprávnení uvedených vo Vyhlásení, je nutný súhlas autora.

Odkaz na túto prácu: Ďurkovič Jakub. *Anketa ČVUT – Admin rozhraní – podpora pro hlavního administrátora*. Bakalárska práca. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

*Velká vdaka patří Ing. Michalovi Valentovi, Ph.D.
za jeho pomoc a ochotu při tvorbě bakalářské práce.
Taktiež by som chcel poďakovať všetkým mojim blíz-
kym, hlavne môjmu otcovi, mame a priateľke, ktorí to
so mnou úspešne vydržali.*

Vyhlásenie

Prehlasujem, že som predloženú prácu vypracoval samostatne a že som uviedol všetky použité informačné zdroje v súlade s Metodickým pokynom o dodržiavaní etických princípov pri príprave vysokoškolských záverečných prác.

Prehlasujem, že som v priebehu príprav a písania záverečnej práce použil nástroje umelej inteligencie. Vygenerovaný obsah som overil. Potvrdzujem, že som si vedomý, že za obsah záverečnej práce plne zodpovedám.

Beriem na vedomie, že sa na moju prácu vzťahujú práva a povinnosti vyplývajúce zo zákona č. 121/2000 Sb., autorského zákona, v znení neskorších predpisov, najmä skutočnosť, že České vysoké učení technické v Prahe má právo na uzavretie licenčnej zmluvy o užití tejto práce ako školského diela podľa § 60 odst. 1 citovaného zákona.

V Prahe dňa 2. októbra 2026

Abstrakt

Táto bakalárska práca prezentuje výsledky rozšírenia administrátorského rozhrania webovej aplikácie Anketa ČVUT. V úvode práce je opísaný existujúci technologický stav rozhrania, ktorý je doplnený o teoretické východiská použitých technológií.

Následne práca opisuje samotné praktické výstupy. Hlavným prínosom je úspešná integrácia modulu pre tvorbu ateliérovej ankety a podpora pre nové typy anketových otázok. Pre superadministrátora bolo vyvinuté nové rozhranie na správu privilegovaných rolí. Na úrovni databázy došlo k refaktorizácii a zdokumentovaniu PL/SQL procedúr a k zmapovaniu úloh v systéme Oracle Scheduler. Výsledné riešenie taktiež zahŕňa vizuálne úpravy, ktoré majú zlepšiť používateľskú skúsenosť pri správe ankiet a používateľov.

Súčasťou práce je aj overenie implementovaných riešení prostredníctvom manuálneho a automatizovaného testovania. V závere sú zhodnotené dosiahnuté výsledky a navrhnuté možné smery ďalšieho rozvoja systému.

Kľúčové slová Anketa ČVUT, administrátorské rozhranie, Kotlin, Spring Boot, PL/SQL, Oracle, Vue.js

Abstract

This bachelor thesis presents the results of the extension of the administrative interface for the Anketa ČVUT web application. The introduction of the thesis describes the existing technological state of the interface, supplemented by the theoretical background of the technologies used.

Subsequently, the work describes the practical outputs themselves. The main contribution is the successful integration of a module for creating studio surveys and support for new types of survey questions. A new interface for managing privileged roles was developed for the superadministrator. At the database level, PL/SQL procedures were refactored and documented, and tasks within the Oracle Scheduler system were mapped. The final solution also includes visual adjustments aimed at improving the user experience in survey and user management.

The thesis also includes the verification of the implemented solutions through manual and automated testing. In the conclusion, the achieved results are evaluated, and potential directions for further development of the system are proposed.

Keywords Survey CTU, admin interface, Kotlin, Spring Boot, PL/SQL, Oracle, Vue.js

Obsah

1	Úvod	1
2	Analýza súčasného technologického stavu	3
2.1	Architektúra systému	3
2.2	Perzistentná vrstva	3
2.2.1	Oracle databáza	4
2.2.2	PL/SQL	4
2.3	Backend	4
2.3.1	Kotlin	5
2.3.2	Spring Boot	5
2.3.3	Architektúra backendu	6
2.3.4	API	6
2.4	Frontend	7
2.4.1	JavaScript	7
2.4.2	Vue.js	7
2.4.3	Nuxt	8
2.4.4	Architektúra	8
2.5	Autentifikácia	8
2.5.1	SSO a Shibboleth	8
2.5.2	JWT	9
3	Rozbor požiadaviek a návrh riešenia	11
3.1	Ateliérová anketa v administrátorskom rozhraní	11
3.1.1	Analýza požiadavky	11
3.1.2	Návrh riešenia	12
3.2	Správa privilegovaných používateľov	13
3.2.1	Analýza požiadavky	13
3.2.2	Návrh riešenia	13
3.3	Nový typ otázok	13
3.3.1	Analýza požiadavky	13
3.3.2	Návrh riešenia	14
3.4	RefaktORIZÁCIA A dokumentácia PL/SQL kódu	15
3.4.1	Analýza požiadavky	15
3.4.2	Návrh riešenia	15
3.5	Dokumentácia Oracle Scheduler úloh na úrovni databázy	15
3.5.1	Analýza požiadavky	15

3.5.2	Návrh riešenia	15
3.6	Úprava rozhrania pre správu používateľov	16
3.6.1	Analýza požiadavky	16
3.6.2	Návrh riešenia	16
3.7	Úprava otázok a zoznamov otázok	16
3.7.1	Analýza požiadavky	16
3.7.2	Návrh riešenia	17
4	Implementácia požiadaviek	19
4.1	Ateliérová anketa v administrátorskom rozhraní	19
4.1.1	Úpravy v databáze Oracle	19
4.1.2	Úpravy na backende	20
4.1.3	Úpravy na frontende	22
4.2	Správa privilegovaných používateľov	24
4.2.1	Úpravy na backende	24
4.2.2	Úpravy na frontende	26
4.3	Nový typ otázok	27
4.3.1	Úpravy na backende	27
4.3.2	Úpravy na frontende	28
4.4	RefaktORIZÁCIA A dokumentácia PL/SQL kódu	29
4.4.1	MERGE_ALL_UMAP_ROLES	29
4.4.2	MERGE_UMAP_ROLE	30
4.4.3	MOVE_UNSET_ROLES_FROM_USERMAP	32
4.5	Dokumentácia Oracle Scheduler úloh na úrovni databázy	33
4.6	Úprava rozhrania pre správu používateľov	35
4.7	Úprava otázok a zoznamov otázok	36
5	Testovanie	37
5.1	Manuálne testovanie	37
5.2	Jednotkové testovanie	37
5.3	Integračné testovanie	39
6	Záver	41
A	Zdrojové kódy	43
	Bibliografia	45
	Obsah príloh	49

Zoznam obrázkov

4.1	Zmeny v modálnom okne pri vytváraní otázky	28
4.2	Diagram popisujúci dané PL/SQL procedúry	32
4.3	Stránka pre správu administrátorov	35

Zoznam tabuliek

Zoznam výpisov kódu

4.1	Nová funkcia <code>NEW_ATELIER_SURVEY</code> v balíku <code>FA_ATELIERY</code> na riadku 18	20
4.2	Ústrižok pôvodnej implementácie metódy <code>createSurvey</code>	21
4.3	Aktualizovaný <code>SurveyCreateDTO</code> v <code>surveys-api.yml</code>	21
4.4	Ústrižok z aktualizovanej metódy <code>createSurvey</code>	22
4.5	Zmena získavania typov ankiet	22
4.6	Nové funkcie v sekcii <code>computed</code>	23
4.7	Pridaný atribút do tela HTTP požiadavky	23
4.8	Ústrižok z entity <code>AllowedTeacher</code>	24
4.9	Rozhranie <code>AllowedTeacherRepository</code>	24
4.10	Trieda <code>AllowedTeacherMapper</code>	25
4.11	Prvotná implementácia konkrétnej triedy <code>AllowedTeacherService</code>	25
4.12	Prvotná implementácia triedy <code>AllowedTeacherController</code>	26
4.13	Entita reprezentujúca odpovedovú možnosť	27
4.14	Nové atribúty v entite <code>Question</code>	28
4.15	Ústrižok z implementovaných zmien v <code>MERGE_UMAP_ROLE</code>	31
5.1	Jednotkový test na skontrolovanie získania všetkých učiteľov	38

5.2	Integračný test kontroluje správne namapovanie GET požiadavku	39
-----	---	----

Zoznam skratiek

API	Application programming interface
CRUD	Create, read, update and delete
ČVUT	České vysoké učení technické
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPQL	Java Persistence query language
JSON	JavaScript object notation
JVM	Java Virtual Machine
JWE	JSON Web Encryption
JWS	JSON Web Signature
JWT	JSON Web Token
OAS	OpenAPI Specification
PL/SQL	Procedural Language for SQL
RDBMS	Relational database management system
REST	Representational State Transfer
SAML	Security assertion markup language
SEO	Search engine optimization
SPA	Single-page applications
SQL	Structured Query Language
SSO	Single sign-on

Kapitola 1

Úvod

Anketa ČVUT je webová aplikácia určená na systematický zber spätnej väzby od študentov všetkých fakúlt ČVUT. Takýto hodnotiaci systém prináša pre vzdelávacie inštitúcie množstvo benefitov, z ktorých profitujú všetci v akademickom prostredí. Pre prednášajúcich a vyučujúcich predstavuje relevantný a štruktúrovaný zdroj informácií o kvalite pedagogickej činnosti, ktorý možno využiť na vylepšenie vyučovacích metód a pedagogických prístupov. Študentom systém uľahčuje výber predmetov prostredníctvom prístupu k štatistikám, akými sú úspešnosť či priemerná známka, a zároveň im umožňuje zanechať anonymnú spätnú väzbu.

ČVUT je univerzita s mnohými fakultami a postupom času vznikla potreba systematickej správy jednotlivých ankiet. Výsledkom toho bolo vytvorenie administrátorského rozhrania Ankety ČVUT. Toto rozhranie poskytuje mnoho užitočných funkcií. Medzi funkcie patrí napríklad vytváranie, úprava a mazanie rôznych typov anketových otázok. Tieto otázky dokážu administrátori následne zoskupovať do zoznamov otázok, ktoré dokážu upravovať a následne priradiť pre špecifickú anketu. Systém umožňuje aj založenie, správu a vyhodnotenie ankety. V administrátorskom rozhraní existuje hierarchia privilégii, rozlišuje sa medzi adminom a superadministrátorom. Táto rola umožňuje súčasnému superadministrátorovi, Ing. Michalovi Valentovi, Ph.D., spravovať systém na vyššej úrovni.

Cieľom tejto bakalárskej práce je zlepšiť súčasný technologický a vizuálny stav administrátorského rozhrania aplikácie Anketa ČVUT. Na základe špecifikovaných požiadaviek, ktoré boli formalizované vo forme úloh v systéme GitLab, je cieľom navrhnuť, implementovať, nasadiť do vývojového a testovacieho prostredia a následne otestovať navrhované riešenia.

Realizované úpravy majú za cieľ zefektívniť správu privilegovaných rolí zo strany superadministrátora, umožniť vytváranie ateliérovej ankety prostredníctvom administrátorského rozhrania, rozšíriť aplikáciu o nové typy otázok a celkovo zlepšiť používateľskú skúsenosť.

Ďalším cieľom bakalárskej práce je refaktorovať PL/SQL procedúry a vytvoriť technickú dokumentáciu, ako aj vypracovať dokumentáciu Oracle Scheduler úloh.

Vývoj ankety a administrátorského prostredia prešiel už mnohými softvérovými tímovými projektmi aj bakalárskymi prácami. Táto bakalárska práca bude nadväzovať na bakalársku prácu Adama Berkeša, ktorá sa zaoberala aktualizáciou a rozšírením backendu [1], a na bakalársku prácu Vojtěcha Hejleka, ktorý navrhol a implementoval rozhranie pre vytvorenie ateliérovej ankety [2].

V kapitole Analýza súčasného technologického stavu budú podrobne analyzované technológie využívané v administrátorskom rozhraní Ankety ČVUT, ktoré tvoria východisko pre ďalšiu realizovanú prácu. Jadro tejto práce bude tvoriť tretia a štvrtá kapitola, v ktorých budú špecifikované a analyzované požiadavky, predstavené návrhy riešení, ich implementácia do existujúceho zdrojového kódu a následné testovanie prostredníctvom jednotkových testov, integračných testov a manuálneho overovania. Záverečná kapitola sa bude venovať vyhodnoteniu splnenia zadania, analýze dosiahnutých výsledkov bakalárskej práce a identifikácii možných smerov ďalšieho vývoja administrátorského rozhrania.

[illegible]

V nasledujúcich podkapitolách budú postupne detailne analyzované všetky tri vrstvy systému.

2.2.1 Oracle databáza

Databázy Oracle predstavujú jedny z najrozšírenejších systémov na správu relačných databáz na svete. Boli vyvinuté spoločnosťou Oracle Corporation v roku 1979 a postupne sa stali jej kľúčovým produktom. V súčasnosti sú databázy Oracle rozsiahlo nasadzované v podnikovom prostredí, najmä vďaka svojej schopnosti efektívne spracovávať, uchovávať a spravovať veľké objemy dát. Medzi ich kľúčové vlastnosti patrí vysoká úroveň zabezpečenia, pokročilé a vo veľkej miere automatizované mechanizmy zálohovania a obnovy dát, ako aj rozsiahla a detailne spracovaná technická dokumentácia [5].

Z uvedených dôvodov je databáza Oracle využívaná aj na potreby celej Ankety ČVUT. Obsah tejto bakalárskej práce sa zameriava na databázové schémy NOVA_ANKETA a ANK_ANKETA. Prítomnosť dvoch odlišných schém je podmienená historickým vývojom systému: pôvodne existovala len schéma ANK_ANKETA, ktorá mala nakonfigurovaný DB_LINK do systémov KOS a Usermap. Neskôr bola navrhnutá a implementovaná nová verzia Ankety ČVUT, pre ktorú bola vytvorená samostatná schéma NOVA_ANKETA.

2.2.2 PL/SQL

PL/SQL predstavuje procedurálne rozšírenie systému Oracle RDBMS, ktoré umožňuje vytvárať procedurálny kód integrovaný s jazykom SQL. Počas procesu kompilácie tento kód databázový systém Oracle prekladá do procedúr, funkcií a balíkov, ktoré sú následne uložené priamo v databáze. Skutočnosť, že PL/SQL aj SQL sú vykonávané v tom istom serverovom procese, prispieva k zvýšeniu celkovej výkonnosti systému, keďže minimalizuje režijné náklady spojené s prepínaním kontextu medzi aplikačným serverom a databázou. Tento prístup zároveň predstavuje efektívny mechanizmus presunu časti logiky a zodpovednosti z aplikačného backendu do databázovej vrstvy, čím sa znižuje záťaž a dosahuje sa centralizovanejšie a konzistentnejšie spracovanie dát [6].

Vzhľadom na to, že systém Anketa ČVUT využíva databázovú platformu Oracle, je možné implementovať množstvo užitočných funkcií a procedúr, ktoré sú zoskupené do balíkov. Tieto balíky zabezpečujú široké spektrum operácií, ktoré by inak museli byť implementované na úrovni backendu aplikácie.

Táto bakalárska práca sa primárne zameriava na analýzu a návrh balíkov FA_ATELIERY a NEW_SEM definovaných v schéme NOVA_ANKETA, ako aj na procedúry spúšťané prostredníctvom Oracle Scheduler úloh v schéme ANK_ANKETA.

2.3 Backend

Druhou vrstvou, ktorou sa táto kapitola bude zaoberať, je backend, často označovaný ako serverová časť aplikácie. Táto vrstva zabezpečuje interakciu s databázou, implementáciu biznis logiky, prijímanie a spracovanie požiadaviek a zohráva kľúčovú úlohu pri ochrane citlivých údajov [7]. V nasledujúcich

podkapitolách bude podrobne analyzovaná práve táto vrstva a technológie využívané pri jej implementácii v kontexte administrátorského rozhrania Ankety ČVUT.

2.3.1 Kotlin

Backend administrátorského rozhrania Ankety ČVUT je implementovaný v programovacom jazyku Kotlin, ktorý predstavuje moderný, staticky typovaný jazyk vyvinutý spoločnosťou JetBrains. Táto spoločnosť ho charakterizuje ako vysoko efektívny a dobre čitateľný jazyk, navrhnutý „od vývojárov pre vývojárov“, so zameraním na zníženie množstva boilerplate kódu a zvýšenie produktivity pri vývoji softvéru. Veľkou výhodou Kotlinu je, že beží na JVM a disponuje plnou interoperabilitou s jazykom Java, čo umožňuje hladkú integráciu oboch jazykov v rámci jedného projektu. Vďaka Kotlin Multiplatform Technology môžu vývojári písať jeden kód pre viacero platforiem, čo umožňuje rýchlejší a flexibilnejší vývoj [8].

2.3.2 Spring Boot

Java Spring Boot je nástroj, ktorý zjednodušuje prácu so Spring Framework, ktorý slúži na vytváranie aplikácií bežiacich na JVM. Jednou z kľúčových predností Spring Bootu je mechanizmus autokonfigurácie. Ten zabezpečuje automatické nastavenie samotného rámca a ďalších komponentov aplikácie na základe prítomných závislostí a konfigurácie definovanej vývojárom.

Spring Boot využíva *opinionated* prístup k vývoju softvéru, čo v praxi znamená, že na základe osvedčených architektonických a implementačných postupov automatizovane určuje predvolenú konfiguráciu a súbor závislostí. Vývojár tak nie je nútený manuálne nastavovať jednotlivé konfiguračné parametre systému, ale môže využívať vopred pripravené moduly. Tieto moduly agregujú všetky nevyhnutné knižnice pre špecifické aplikačné domény, čím sa významne redukuje čas potrebný na inicializáciu projektu a zároveň sa minimalizuje riziko verziovej nekompatibility medzi komponentmi. Spring Boot navyše umožňuje vývoj samostatne spustiteľných aplikácií, ktoré nevyžadujú závislosť od externého webového servera, keďže obsahujú zabudovaný webový server [9].

Aj keď používanie Spring Boot nie je nevyhnutnou podmienkou pri práci so Spring Framework, jeho využitie sa dôrazne odporúča z hľadiska zjednodušenia konfigurácie, zvýšenia produktivity a štandardizácie architektúry aplikácie. Z týchto dôvodov je Spring Boot použitý aj v tomto projekte, konkrétne vo verzii 3.4.2.

2.3.3 Architektúra backendu

Zdrojový kód backendovej časti aplikácie je navrhnutý v súlade s princípmi viacvrstvovej architektúry, v rámci ktorej jednotlivé triedy zohrávajú špecifické, navzájom oddelené funkčné úlohy. Viacvrstvomá architektúra rozdeľuje aplikačný kód do viacerých nezávislých vrstiev, ktoré medzi sebou komunikujú prostredníctvom definovaných rozhraní, čím sa zvyšuje modularita, udržiavateľnosť a testovateľnosť systému [10].

Perzistentnú vrstvu tvoria súbory umiestnené v balíku `repository` a `entity`. Všetky implementované rozhrania tried typu `repository` dedia z rozhrania `JpaRepository`, ktoré poskytuje štandardné CRUD operácie, ako aj možnosť definovať ďalšie databázové operácie prostredníctvom konvencií pomenovania metód. Vďaka tomu nie je nutné implementovať tieto metódy manuálne, pokiaľ postačuje odvodenie dopytov z názvu metódy. V prípadoch, keď takýto mechanizmus nestačí, sa využíva anotácia `@Query`, ktorá umožňuje explicitné špecifikovanie SQL alebo JPQL dopytov. Entitné triedy reprezentujú tabuľky v relačnej databáze, pričom jednotlivé inštancie entít korešpondujú s riadkami v príslušných tabuľkách [11].

Ďalšiu vrstvu tvorí servisná vrstva, kam zaraďujeme triedy označené anotáciou `@Service`. Táto vrstva implementuje biznis logiku aplikácie, riadi logický priebeh spracovania požiadaviek a zabezpečuje oddelenie perzistentnej vrstvy od prezentačnej vrstvy. Zodpovedá za orchestráciu operácií nad dátami, aplikovanie doménových pravidiel a koordináciu volaní jednotlivých komponentov [12].

Prezentačná vrstva zabezpečuje spracovanie HTTP požiadaviek od klientov a je implementovaná pomocou tried anotovaných ako `@Controller`. Táto vrstva definuje REST endpointy, vykonáva validáciu vstupných požiadaviek a vracia odpovede so zodpovedajúcimi HTTP status kódmi v súlade s RESTful princípmi [13]. Súčasťou prezentačnej vrstvy môžu byť aj pomocné mapovacie triedy, ktoré sprostredkujú konverziu medzi entitami a DTO v oboch smeroch, čím sa ďalej podporuje oddelenie vnútorného dátového modelu od externého rozhrania aplikácie.

2.3.4 API

Tento projekt využíva moderný prístup k návrhu a implementácii API, označovaný ako *API-First Approach*. Ide o metodológiu vývoja softvéru, pri ktorej sa v úvodnej fáze formálne špecifikuje API a až následne sa realizuje samotná implementácia aplikačnej logiky [14].

Pri návrhu API sa odporúča dodržiavať špecifikáciu OAS, ktorá predstavuje štandard pre strojovo čitateľný popis rozhrania. V administrátorskom module projektu sú tieto rozhrania umiestnené v adresári `anketa-admin-api` a sú definované vo formáte YAML.

Spustením príkazu `./gradlew :anketa-admin-api:openApiGenerate` sa na základe uvedených YAML špecifikácií automatizovane generujú abstraktné triedy API. Tieto triedy následne slúžia ako базové triedy pre implementáciu kontrolérov. Uvedený prístup umožňuje jednoznačnú, konzistentnú a ľahko čitateľnú definíciu všetkých endpointov, ktoré daný projekt poskytuje, pričom YAML špecifikácia predstavuje jediný zdroj pravdy (z angl. single source of truth) pre celé API.

2.4 Frontend

Poslednou vrstvou, ktorou sa táto kapitola bude zaoberať, je frontend, označovaný aj ako klientska časť aplikácie. Vývoj frontendu sa primárne sústreďuje na optimalizáciu používateľského zážitku. Cieľom je navrhovať a implementovať interaktívne komponenty, s ktorými môžu používatelia aktívne pracovať. Na základe zaznamenaných používateľských interakcií môže frontend následne iniciovať volania rozhrania API smerom k backendovej časti systému [15].

2.4.1 JavaScript

Pre frontendovú časť tohto projektu bol zvolený jazyk JavaScript, ktorý predstavuje esenciálnu súčasť moderného webového vývoja. JavaScript je vysokoúrovňový programovací jazyk, vytvorený v roku 1995, ktorý sa primárne využíva na tvorbu webových aplikácií v kombinácii s HTML a CSS. Zatiaľ čo HTML zabezpečuje štruktúru webovej stránky a CSS definuje jej vizuálnu prezentáciu, JavaScript umožňuje implementáciu interaktívnych prvkov a dynamického správania používateľského rozhrania [16].

2.4.2 Vue.js

Tento projekt ďalej využíva framework Vue.js, konkrétne jeho verziu Vue 3. Vue.js je open-source JavaScript framework, ktorý umožňuje vývojárom vytvárať interaktívne a vizuálne bohaté používateľské rozhrania. Nadväzuje na technológie HTML, CSS a JavaScript, pričom ich rozširuje prostredníctvom intuitívne navrhnutého API a rozsiahlej, systematicky spracovanej dokumentácie.

Medzi kľúčové vlastnosti Vue.js patrí deklaratívny prístup k vykresľovaniu používateľského rozhrania, v rámci ktorého vývojár špecifikuje vzhľad a správanie rozhrania na základe aktuálneho stavu dát. Ďalším kľúčovým konceptom je komponentovo orientovaná architektúra, ktorá podporuje opätovnú použiteľnosť kódu, modularitu a lepšiu udržiavateľnosť softvérových riešení [17].

2.4.3 Nuxt

Nuxt je framework určený na vývoj webových aplikácií postavených na knižnici Vue.js. V tomto projekte sa konkrétne využíva jeho verzia Nuxt 3. Jednou z kľúčových výhod Nuxt je mechanizmus *file-system routing*, ktorý umožňuje automatizované generovanie aplikačných trás na základe štruktúry adresára **pages**. Tento prístup zvyšuje prehľadnosť a udržiavateľnosť kódu, keďže adresárová hierarchia priamo reflektuje štruktúru používateľského rozhrania.

Medzi ďalšie významné prínosy využívania Nuxt patrí zlepšený výkon a optimalizácia pre vyhľadávače, teda spôsob, akým vyhľadávacie nástroje interpretujú a indexujú obsah danej webovej stránky. Framework ďalej podporuje viacero režimov renderovania, vrátane generovania statických webových stránok, vykresľovania na strane servera a tvorby single-page aplikácií [18].

2.4.4 Architektúra

V prípade tohto projektu ide o webovú aplikáciu typu SPA, teda o aplikáciu, ktorá načítava iba jednu HTML stránku a následne na základe asynchrónnych volaní API dynamicky aktualizuje jej obsah bez potreby úplného znovunačítania stránky. Tento prístup zvyčajne prispieva k zvýšeniu výkonu a plynulosti používateľskej interakcie, keďže eliminuje opakované načítavanie celého dokumentu. Na druhej strane má negatívny vplyv na SEO, čo však v kontexte interného systému, akým je administrátorské rozhranie Ankety, nepredstavuje problém, keďže optimalizácia pre vyhľadávače nie je v tomto prípade požiadavkou [19].

2.5 Autentifikácia

2.5.1 SSO a Shibboleth

Na začiatku autentifikačného procesu sa využíva mechanizmus Single Sign-On. SSO predstavuje autentifikačný proces, ktorý používateľom umožňuje získať prístup k viacerým službám na základe jednorazového prihlásenia. V podmienkach ČVUT ide napríklad o prístup do systému Ankety, študijného informačného systému KOS, e-learningovej platformy Moodle a ďalších služieb [20].

V rámci tohto projektu je na implementáciu SSO použitý systém Shibboleth, open-source softvér primárne určený na realizáciu SSO prostredníctvom protokolu SAML. Pri prihlasovaní do administrátorského rozhrania sa Shibboleth pokúša autentifikovať používateľa pomocou univerzitného Identity Providera. Ak je autentifikácia úspešná, Shibboleth vloží do hlavičky HTTP požiadavky identifikátor používateľa a na základe tohto identifikátora backendová časť overí, či sa daný používateľ nachádza v databáze administrátorov. Ak áno, vygeneruje pre neho JWT, ktorý bude rozobraný v ďalšej podkapitole. Samotný Shibboleth sa tak podieľa výlučne na úvodnom overení identity, všetka

ďalšia komunikácia medzi klientom a serverom je zabezpečená prostredníctvom JWT [21].

2.5.2 JWT

Na autentifikáciu používateľov sa v tomto projekte využíva bezstavová autentifikácia založená na mechanizme JWT. JWT predstavuje štandardizovaný textový formát správ určený na kompaktný a bezpečný prenos informácií medzi stranami.

V základnej podobe sa JWT skladá z dvoch kľúčových komponentov. Prvým je payload, t. j. dátová časť tokenu, ktorá nesie samotné aplikačné údaje a tvrdenia. Druhým komponentom je hlavička, ktorá má podobu JSON objektu obsahujúceho metadáta o spôsobe spracovania tokenu, najmä o použitom algoritme a type tokenu, ako aj informácie súvisiace s interpretáciou payloadu.

Aby sa zabezpečila integrita dát, JWT sa štandardne kryptograficky podpisuje pridaním tretej časti (podpisu), čím vzniká JWS, prípadne sa priamo šifruje do podoby JWE. Vďaka týmto mechanizmom kryptografického zabezpečenia a overiteľnosti môže prijímateľ validovať integritu a pôvod prenášaných dát, a tým získať istotu, že údaje neboli počas prenosu modifikované a pochádzajú z dôveryhodného zdroja.

Na generovanie a verifikáciu týchto tokenov sa používa knižnica JJWT, ktorá nahrádza manuálnu realizáciu JSON serializácie a Base64URL kódovania abstraktnou vrstvou vo forme jednoduchých a štandardizovaných API volaní [22].

Rozbor požiadaviek a návrh riešenia

Táto kapitola predstavuje prvú časť jadra tejto práce. Cieľom bakalárskej práce je rozšírenie a aktualizácia administrátorského rozhrania Ankety ČVUT na základe nových požiadaviek, ktoré vznikli od posledného nasadenia systému. Uvedené požiadavky boli zozbierané a následne formalizované vo forme GitLab požiadaviek superadministrátorom Ankety ČVUT, Ing. Michalom Valentom, Ph.D. V tejto kapitole budú analyzované jednotlivé požiadavky a predstavené návrhy ich riešení, zatiaľ čo v nasledujúcej kapitole bude opísaná implementácia.

3.1 Ateliérová anketa v administrátorskom rozhraní

3.1.1 Analýza požiadavky

Na základe požiadaviek Fakulty architektúry ČVUT vytvoril Vojtěch Hejlek v rámci svojej bakalárskej práce nový typ ankety určený pre ateliérovú výučbu [2]. Jedným z výstupov tejto práce je PL/SQL balík FA_ATELIERY uložený v databáze Oracle v schéme NOVA_ANKETA. Tento balík obsahuje niekoľko procedúr, ktoré zabezpečujú vytvorenie uvedeného typu ankety. Dokumentácia k správaniu poradiu volania týchto procedúr bola poskytnutá vedúcim bakalárskej práce.

Na základe podrobnej analýzy existujúcej implementácie procesu vytvárania štandardných ankiet naprieč všetkými vrstvami webovej aplikácie, ako aj preštudovania príslušnej dokumentácie, boli identifikované viaceré problémy, ktoré je potrebné vyriešiť.

Prvým problémom je skutočnosť, že v administrátorskom prostredí už existuje mechanizmus na vytváranie štandardných ankiet. Tento mechanizmus je potrebné vhodne integrovať s novým typom ankety bez nutnosti výrazných zá-

sahov do existujúcej aplikačnej logiky. V triede `ISurveyServiceImpl` sa nachádza metóda `createSurvey`, ktorá zodpovedá za vytváranie ankiet tak, že volá existujúce PL/SQL funkcie zabezpečujúce vytvorenie štandardných ankiet. Na základe vstupných parametrov sa volá buď funkcia `NEW_SURVEY_COURSE`, alebo `NEW_SURVEY_FACULTY` z balíka `NEW_SEM`. Pre ateliérovú anketu však obdobná súhrnná funkcia neexistuje. K dispozícii je iba viacero čiastkových procedúr, ktoré je potrebné skombinovať do jednej koherentnej funkcie. Rozšírenie systému o nový typ ankety si teda vyžaduje v prvom kroku návrh a implementáciu PL/SQL funkcie, ktorá v rámci jedného behu sekvenčne spustí všetky potrebné čiastkové procedúry, a v druhom kroku doplnenie volania tejto funkcie do metódy `createSurvey` na backendovej vrstve.

Druhým problémom je samotný návrh a implementácia uvedenej novej PL/SQL funkcie. V aktuálnej dokumentácii, ktorá opisuje proces vytvárania ateliérovej ankety pomocou viacerých na sebe nezávislých transakcií, sa medzi krokom 1 a krokom 2 nachádza manuálny krok spočívajúci v nahratí novej aktuálnej mapovacej matice `FA_ATELIERY_MAP` do databázy. Pri procedúrach `NEW_SURVEY_COURSE` a `NEW_SURVEY_FACULTY` takýto manuálny zásah nebol potrebný, čo implikuje potrebu navrhnuť nový, plne automatizovaný spôsob zaobchádzania s mapovacou maticou tak, aby bolo možné integrovať celý proces vytvárania ateliérovej ankety do jedinej, konzistentnej databázovej transakcie.

Taktiež je nevyhnutná zmena na frontende, pretože aktuálne poskytuje možnosti iba pre vytvorenie predmetovej, fakultnej alebo kombinovanej predmetovej a fakultnej ankety.

3.1.2 Návrh riešenia

Na základe diskusie so superadministrátorom bola navrhnutá nasledujúca forma riešenia požiadavky. V balíku `FA_ATELIERY` bude vytvorená nová PL/SQL funkcia `NEW_ATELIER_SURVEY`. Táto funkcia bude mať identický počet vstupných aj výstupných parametrov ako existujúce funkcie určené na vytváranie ankiet, s cieľom zabezpečiť bezproblémovú integráciu do backendovej metódy `createSurvey`.

Nová funkcia bude predpokladať existenciu a aktuálnosť mapovacej matice `FA_ATELIERY_MAP`. V prípade, že táto matica bude prázdna, funkcia vráti chybový stav. O predpoklade existencie a aktuálnosti mapovacej matice bude používateľ informovaný prostredníctvom informačného dialógového okna zobrazeného pred vytvorením ateliérovej ankety.

Implementácia tejto požiadavky bude vyžadovať zásah do všetkých troch aplikačných vrstiev. Na frontendovej úrovni bude potrebné doplniť ateliérovú anketu, pričom táto možnosť bude dostupná výlučne v prípade, ak je zvolená Fakulta architektúry, keďže pre ostatné fakulty sa tento typ ankety nevyužíva.

3.2 Správa privilegovaných používateľov

3.2.1 Analýza požiadavky

Cieľom tejto požiadavky je vytvorenie novej stránky v administrátorskom rozhraní, ktorá bude určená výhradne pre potreby superadministrátora. Táto stránka bude slúžiť na správu privilegovaných rolí. V schéme NOVA_ANKETA sa nachádza tabuľka RES_ALLOWED_TEACHERS_PRIV, ktorá uchováva aktuálny zoznam privilegovaných rolí. Tieto záznamy sú priebežne synchronizované so systémom Usermap pomocou PL/SQL procedúr, ktorých podrobnejší opis bude uvedený v ďalšej časti tejto práce.

V uvedenej tabuľke existuje viacero stĺpcov, ktoré musí superadministrátor v mimoriadnych prípadoch manuálne aktualizovať. Absencia intuitívneho používateľského rozhrania podstatne komplikuje prácu s touto tabuľkou, keďže jediná možnosť vykonávania zmien v súčasnosti spočíva v použití SQL dopytov priamo nad tabuľkou. Z tohto dôvodu je potrebné doplniť novú stránku s názvom *Privilegované roly*, ktorá bude zobrazovať tabuľku s aktuálnym obsahom a zároveň umožní jednoduché vyhľadávanie, triedenie a vykonávanie CRUD operácií s jednotlivými záznamami.

3.2.2 Návrh riešenia

Navrhované riešenie si vyžiada zásah do frontendovej aj backendovej časti aplikácie. Na úrovni frontendu bude potrebné implementovať Vue.js komponent, ktorý zabezpečí odkazovanie na novú stránku, a v adresári pages vytvoriť novú stránku určenú na zobrazovanie a spracovanie príslušného obsahu. Dizajn stránky *Privilegované roly* bol navrhnutý a implementovaný na základe vopred pripraveného návrhu, ktorý bol následne schválený superadministrátorom.

Na úrovni backendu je z dôvodu zachovania existujúcej viacvrstvovej architektúry, ktorá je v aktuálnom kóde už udržiavaná, nevyhnutné vytvoriť všetky príslušné vrstvy, konkrétne triedy typu Repository, Controller, Service, Mapper a Entity. Súčasne bude potrebné aktualizovať API tak, aby reflektovalo novo zavedené operácie a zodpovedajúcu entitu. Okrem toho bude nevyhnutné implementovať aj príslušné jednotkové testy.

3.3 Nový typ otázok

3.3.1 Analýza požiadavky

V semestroch B242 a B251 boli v rámci tímového projektu navrhnuté a implementované nové typy otázok. Doteraz boli k dispozícii iba obrázkové, textové a kombinované (obrázkové a textové) otázky, čo sa v niektorých prípadoch uká-

zalo ako limitujúce. Z tohto dôvodu boli zavedené typy otázok *1 z N odpovedí* a *M z N odpovedí*.

Uvedené typy otázok sú v systéme Anketa ČVUT už implementované vo všetkých aplikačných vrstvách, avšak administrátorské rozhranie ich zatiaľ nepodporuje. Realizácia tejto požiadavky bude vyžadovať naštudovanie výstupov tímového projektu a na základe toho následnú integráciu nových typov otázok do administrátorského rozhrania.

3.3.2 Návrh riešenia

Na základe podrobnej analýzy výstupov tímového projektu a aktuálnej implementácie administrátorského rozhrania bol vypracovaný nasledujúci návrh riešenia, ktorý bol konzultovaný a odsúhlasený so superadministrátorom.

Z hľadiska frontendovej časti je potrebný zásah do okna určeného na modifikáciu a vytváranie otázok. Existujúce typy otázok *obrázková*, *textová* a *oboje* budú zmenené na nasledujúce kategórie: *obrázková*, *textová*, *obrázková + textová*, *výber jednej* a *výber viacerých*.

Na základe dodatočnej požiadavky superadministrátora bude upravená aj viditeľnosť prepínačov *primárna otázka* a *povinná otázka* podľa týchto pravidiel:

- ak je zvolený typ otázky **obrázková**, prepínače *primárna* aj *povinná* budú viditeľné
- ak je zvolený typ **obrázková + textová**, oba prepínače nebudú viditeľné a pri vytváraní otázky budú príslušné hodnoty automaticky nastavené na hodnotu **false**
- pri typoch **textová**, **výber jednej**, **výber viacerých** bude prepínač *povinná* viditeľný, zatiaľ čo prepínač *primárna* nebude viditeľný a jeho hodnota bude nastavená na **false**.

Na úrovni backendu bude potrebné upraviť entitu reprezentujúcu otázku, keďže v nej chýbajú nové polia zodpovedajúce novým typom otázok. Súčasťou tejto úpravy musí byť aj modifikácia príslušného API.

Pri analýze databázovej implementácie nových typov otázok bol identifikovaný problém súvisiaci s chýbajúcimi stĺpcami, ktoré sú pre ich fungovanie nevyhnutné. Tento fakt pôvodne naznačoval, že táto kľúčová časť nebola v rámci tímového projektu realizovaná. Následnou verifikáciou sa však ukázalo, že išlo o technickú nezrovnalosť: nástroj Flyway aktualizoval databázovú schému na zastaranú verziu, v ktorej stĺpce `IS_MULTIANSWER_OPTION` a `HAS_MULTICHOICE_ANSWER` ešte neboli prítomné.

3.4 Refaktorizácia a dokumentácia PL/SQL kódu

3.4.1 Analýza požiadavky

Ďalšia požiadavka je zameraná výlučne na perzistentnú vrstvu, konkrétne na PL/SQL procedúry. Ako už bolo v tejto bakalárskej práci uvedené, v databáze Oracle systému Anketa ČVUT sa nachádza množstvo PL/SQL balíkov, funkcií a procedúr, ktoré zefektívňujú a zjednodušujú prácu s tabuľkami. Táto požiadavka sa špecificky zameriava na dokumentáciu procedúr `MERGE_UMAP_ROLE`, `MOVE_UNSET_ROLES_FROM_USERMAP` a `MERGE_ALL_UMAP_ROLES` uložených v databázovej schéme `NOVA_ANKETA`.

3.4.2 Návrh riešenia

Na základe diskusie bol navrhnutý nasledujúci metodický postup. V prvej fáze riešenia požiadavky bude potrebné detailne porozumieť existujúcim procedúram a systematicky ich zdokumentovať. Druhá fáza realizácie sa bude zameriavať na refaktorizáciu existujúcich zdrojových kódov procedúr. Výstupy prvej a druhej fázy budú odprezentované a v prípade, že nebudú vznesené pripomienky, budú procedúry overené na testovacích dátach. Po úspešnom ukončení testovania budú na platformu GitLab nahrané príslušné dokumentačné materiály a zdrojové kódy, ktoré budú superadministrátorom následne zaradené do produkčnej databázy systému Anketa ČVUT.

3.5 Dokumentácia Oracle Scheduler úloh na úrovni databázy

3.5.1 Analýza požiadavky

Cieľom tejto požiadavky je vypracovať systematickú dokumentáciu aktuálne používaných Oracle Scheduler úloh súvisiacich s databázou systému Anketa. Pre tieto Oracle Scheduler úlohy v súčasnosti neexistuje žiadna forma formálnej dokumentácie, čo vedie k nejasnostiam ohľadom presného obsahu a periodicity spúšťaných procesov. Výstup tejto požiadavky má poskytnúť superadministrátorovi podklad pre efektívnejšiu správu a organizáciu databázového prostredia.

3.5.2 Návrh riešenia

Kvôli nedostatku dokumentácie k existujúcim Oracle Scheduler úlohám je potrebné zistiť, ktoré úlohy sú aktuálne aktívne a skutočne sa vykonávajú.

Na identifikáciu a analýzu existujúcich Oracle Scheduler úloh bude použitý nástroj Oracle SQL Developer, ktorý poskytuje vhodnejšie prostredie na prácu

s databázou Oracle v porovnaní s nástrojom DataGrip od spoločnosti JetBrains [23]. Oracle SQL Developer zároveň umožňuje detailnú inšpekciu Oracle Scheduler úloh, vrátane metadát o ich vytvorení, harmonograme spúšťania a, samozrejme, aj zdrojového kódu, ktorý je prostredníctvom nich vykonávaný.

Na základe takto získaných údajov bude vytvorený súbor vo formáte Markdown, v ktorom budú jednotlivé Oracle Scheduler úlohy systematicky a podrobne zdokumentované. Dokument bude zároveň navrhnutý tak, aby poskytoval priestor na konzistentné dopĺňanie nových Oracle Scheduler úloh v budúcnosti.

3.6 Úprava rozhrania pre správu používateľov

3.6.1 Analýza požiadavky

Táto požiadavka sa zaoberá vizuálnym a funkčným rozšírením používateľského rozhrania stránky určenej na správu administrátorov ankety. Prvou požadovanou zmenou je pridanie nového stĺpca *Poznámka*. Tento stĺpec bude plniť výlučne informačnú funkciu pre superadministrátora a jeho obsah bude možné upravovať.

Ďalšou úpravou je implementácia funkcie vyhľadávania v zozname administrátorov a zavedenie možnosti triedenia podľa jednotlivých stĺpcov.

Primárnym cieľom týchto úprav je zvýšiť efektivitu práce superadministrátora pri správe jednotlivých administrátorov ankiet.

3.6.2 Návrh riešenia

Na základe revízie databázovej štruktúry bolo zistené, že už existuje stĺpec určený na ukladanie poznámok, avšak ten sa v súčasnosti nevyužíva ani na úrovni backendu, ani na úrovni frontendovej aplikácie. Na strane backendu je potrebné aktualizovať príslušné API vrátane rozšírenia návratových dátových štruktúr tak, aby zahŕňali aj uvedené pole na poznámky. Na strane frontendu je nevyhnutné rozšíriť tabuľkové zobrazenie o nový stĺpec *Poznámka* a zároveň v dialógovom okne určenom na úpravu záznamu doplniť funkcionality umožňujúcu zobraziť a upravovať hodnotu tohto poľa.

3.7 Úprava otázok a zoznamov otázok

3.7.1 Analýza požiadavky

Posledné dve GitLab požiadavky boli zlúčené do jednej podkapitoly, keďže sa zaoberajú veľmi podobnou problematikou. Jadrom riešeného problému sú entity reprezentujúce otázku a zoznam otázok.

Na stránke *Zoznamy otázok* je požadované doplniť identifikátor otázky z databázy s cieľom zefektívniť a urýchliť jej identifikáciu. Zároveň je potrebné analyzovať, ktoré vlastnosti zoznamu otázok môžu byť modifikované v závislosti od jeho histórie a aký vplyv môže mať zmena alebo odstránenie celej sady otázok na všetky s ňou prepojené otázky.

Analogická analýza je potrebná aj na úrovni jednotlivých otázok, konkrétne je nutné preskúmať a zdokumentovať dôsledok odstránenia otázky na ostatné časti systému a súvisiace entity.

3.7.2 Návrh riešenia

Na základe konzultácie so superadministrátorom bol stanovený nasledujúci postup. Zoznamy otázok, ktoré nie sú priradené k žiadnej ankete, bude možné vymazať; v opačnom prípade sa zobrazí varovné hlásenie upozorňujúce, že zmazanie danej sady otázok nie je možné.

Pri aktivácii tlačidla *Zmeniť* sa po novom zobrazí varovanie, ktoré bude správcu informovať, že príslušná sada otázok môže byť využívaná v iných anketach, o ktorých nemusí mať prehľad, a zmena tak môže ovplyvniť výsledné správanie týchto ankiet. Súčasťou varovania bude odporúčanie, aby si správca predmetný zoznam otázok najskôr duplikoval, čím sa minimalizuje riziko nepredvídateľného správania systému.

Analogický mechanizmus bude aplikovaný aj pri mazaní a úprave jednotlivých otázok. Konkrétne, otázka nebude môcť byť vymazaná, pokiaľ je priradená k niektorému zoznamu otázok. Pri úprave otázky sa zobrazí varovanie, že zmena môže mať vplyv na príslušné zoznamy otázok a následne aj na ankety, v ktorých sa daná otázka vyskytuje.

Implementácia požiadaviek

Táto kapitola sa venuje samotnej implementácii požiadaviek zadefinovaných v predchádzajúcej časti práce. Jej štruktúra logicky kopíruje kapitolu o rozbere a návrhu riešenia. Každá podkapitola postupne detailne opisuje konkrétne programátorské zásahy a úpravy naprieč všetkými vrstvami aplikácie, od databázy cez backend až po používateľské rozhranie.

4.1 Ateliérová anketa v administrátorskom rozhraní

4.1.1 Úpravy v databáze Oracle

Na základe vykonanej analýzy a návrhu riešenia bola v balíku FA_ATELIER definovaná nová PL/SQL operácia s názvom NEW_ATELIER_SURVEY. Táto operácia bola deklarovaná ako funkcia z dôvodu zachovania konzistencie s ekvivalentnými operáciami určenými na vytváranie predmetových a fakultných ankiet, ktoré sú takisto implementované ako funkcie. Voľba funkcie namiesto procedúry vyplýva zo špecifik jazyka PL/SQL, v ktorom procedúry neposkytujú návratovú hodnotu. Táto skutočnosť by predstavovala obmedzenie pri volaní z backendovej vrstvy aplikácie, keďže je nevyhnutné explicitne získať informáciu o výsledku vykonania danej operácie.

```

1  create or replace PACKAGE FA_ATELIERY as
2      --zkontroluje jestli jsou data v tabulce FA_ATELIERY_MAP v poradku
3  PROCEDURE validate_table_FA_ATELIERY_MAP;
4      --prida nove ateliery, které se nenachazi v tabulce
4      ↪ FA_ATELIERY_AS_COURSE ale nachazi se v FA_ATELIERY_MAP
4      ↪ (poskytnuta matice). Bereme podle kodu 'ATL-___'
5  PROCEDURE add_new_ateliers;
6      --upravi nazev stavajiciho atelieru dle kodu atelieru
7  PROCEDURE modify_existing_ateliers;
8      --zaobaluje procedury add_new_ateliers a modify_existing_ateliers
9  PROCEDURE prepare_ateliers;
10     --namapuje studenty na ateliery ve kterých studují, naplní tabulku
10     ↪ FA_ENROLLED_STUDENTS_IN_ATELIERS
11  PROCEDURE add_students_to_ateliers;
12     --namapuje ucitele na ateliery ve kterých uci, naplní tabulku
12     ↪ TEACHER_IN_COURSE_SURVEY
13  PROCEDURE add_teachers_to_ateliers(p_semester_code varchar);
14     --priradi spravny pocet znamek k atelierum v tabulce RES_CUR_SUM,
14     ↪ procedura pouze upravuje znamky, nevyhodnocuje ateliery. Je
14     ↪ treba ji poustet az po COURSE_EVAL_CREATE (vyhodnoceni ankety)
15  PROCEDURE evaluate_grades_in_ateliers(p_semester_code varchar);
16     -- Vytvori novou atelierovou anketu pro dany fakultu a semestr.
17     -- !!! FA_ATELIERY_MAP musi byt naplnena mapovaci matici pro
17     ↪ aktualni semestr PRED spustenim !!!
18  FUNCTION new_atelier_survey(p_faculty IN NUMBER,
18     ↪ p_semester_code_new IN VARCHAR2) RETURN VARCHAR;
19
20 END FA_ATELIERY;

```

■ **Výpis kódu 4.1** Nová funkcia NEW_ATELIER_SURVEY v balíku FA_ATELIERY na riadku 18

V jazyku PL/SQL sa logika funkcií a procedúr štandardne definuje v tele balíka (angl. *package body*), pričom samotný balík slúži primárne ako verejné rozhranie [24]. Z tohto dôvodu je potrebné vykonať úpravu v tele balíka FA_ATELIERY, kde je implementovaná logika ostatných procedúr, ktoré vytvoril Vojtěch Hejlek v rámci svojej bakalárskej práce.

4.1.2 Úpravy na backende

V dôsledku úspešnej integrácie novej ateliérovej PL/SQL funkcie boli potrebné len minimálne zmeny na úrovni backendu. V triede `ISurveyServiceImpl` bolo, ako už bolo uvedené v analytickej časti, nutné upraviť metódu `createSurvey`, ktorá v pôvodnej implementácii umožňovala vytvárať ankety výlučne pre fakulty a predmety.

```

1 override fun createSurvey(model: SurveyCreateDto):
  ↳ SurveyCreateResultDto {
2     ...
3     val functionStr = if (model.coursal) {
4         "{? = call NOVA_ANKETA.NEW_SEM.NEW_SURVEY_COURSE(?, ?)}"
5     } else {
6         "{? = call NOVA_ANKETA.NEW_SEM.NEW_SURVEY_FACULTY(?, ?)}"
7     }
8     ...
9 }

```

■ **Výpis kódu 4.2** Ústrižok pôvodnej implementácie metódy createSurvey

Objekt SurveyCreateDTO, ktorý predstavuje DTO generované na základe OpenAPI špecifikácie, pôvodne obsahoval atribút určený výlučne na rozlíšenie medzi fakultnou a predmetovou anketou. Na tento účel sa používal logický atribút `coursal`.

V nadväznosti na rozšírenie počtu typov ankiet na tri sa pôvodný model, založený na jedinom logickom indikátore, ukázal ako nepostačujúci, a vznikla preto požiadavka na doplnenie ďalšieho príznaku. Z tohto dôvodu bol do objektu pridaný nový logický atribút `atelier`. Alternatívnym riešením uvedenej zmeny by bolo odstránenie atribútu `coursal` a jeho nahradenie enumerátorom, ktorý by explicitne určoval konkrétny druh ankety prostredníctvom textového označenia. Tento prístup je vhodné zvážiť v prípade výrazného nárastu počtu typov ankiet, v blízkej budúcnosti sa však takýto scenár nepovažuje za pravdepodobný.

```

1 SurveyCreateDto:
2   type: object
3   required:
4     - facultyId
5     - semesterCode
6     - coursal
7     - atelier
8   properties:
9     facultyId:
10      type: integer
11      format: int64
12     semesterCode:
13      type: string
14     coursal:
15      type: boolean
16     atelier:
17      type: boolean

```

■ **Výpis kódu 4.3** Aktualizovaný SurveyCreateDTO v surveys-api.yml

Po vykonaní príkazu `./gradlew :anketa-admin-api:openApiGenerate` sa objekt `SurveyCreateDto` aktualizuje o nový atribút, čo umožní modifikovať metódu `createSurvey` tak, aby bola schopná vyvolať databázovú funkciu `NEW_ATELIER_SURVEY`.

```

1  override fun createSurvey(model: SurveyCreateDto):
    ↳ SurveyCreateResultDto {
2      ...
3      val functionStr = if (model.coursal && model.atelier) {
4          "{? = call NOVA_ANKETA.FA_ATELIERY.NEW_ATELIER_SURVEY(?,
            ↳ ?)}"
5      } else if (model.coursal) {
6          "{? = call NOVA_ANKETA.NEW_SEM.NEW_SURVEY_COURSE(?, ?)}"
7      } else {
8          "{? = call NOVA_ANKETA.NEW_SEM.NEW_SURVEY_FACULTY(?, ?)}"
9      }
10     ...
11 }

```

■ **Výpis kódu 4.4** Ústrižok z aktualizovanej metódy `createSurvey`

4.1.3 Úpravy na frontende

V rámci úprav na úrovni frontendovej vrstvy boli realizované zmeny v súbore `AddSurveyModal.vue`, ktorý predstavuje implementáciu dialógového komponentu zobrazovaného pri vytváraní novej ankety. Bolo potrebné modifikovať spôsob získavania dostupných typov ankiet pre konkrétnu fakultu, keďže doteraz všetky fakulty využívali rovnakú sadu typov ankiet. Nový ateliérový typ ankety je však určený výhradne pre Fakultu architektúry, a preto bolo nevyhnutné zaviesť mechanizmus dynamického načítavania typov ankiet na základe aktuálne zvolenej fakulty.

```

1  <v-select v-model="surveyType" label="Typ ankety"
    ↳ :items="Object.values(surveyTypes)" :rules="[rules.required]" />
2
3  <!-- After change -->
4
5  <v-select v-model="surveyType" label="Typ ankety"
    ↳ :items="availableSurveyTypes" :rules="[rules.required]" />

```

■ **Výpis kódu 4.5** Zmena získavania typov ankiet

Logika funkcie `availableSurveyTypes` spolu s pomocnými funkciami bola umiestnená do sekcie `computed`, čo je funkcionálna funkcia frameworku `Vue.js` určená na odvodenie a získavanie hodnôt z reaktívneho stavu. Na rozdiel od bežnej

JavaScript funkcie na získavanie dát, ktorá musí pri každom prekreslení komponentu hodnotu znova prepočítať, využitie `computed` predstavuje efektívnejší prístup, keďže výsledné hodnoty sú ukladané do cache a znovu sa prepočítajú len v prípade, že dôjde k zmene niektorej z ich závislostí [25].

```
1 computed: {  
2   ...  
3  
4   selectedFaculty() {  
5     return this.faculties.find(f => f.orgUnitCode ===  
6       ↪ this.faculty.orgUnitCode);  
7   },  
8   isFacultyFA() {  
9     return this.selectedFaculty?.shortNameCz === 'FA';  
10  },  
11  availableSurveyTypes() {  
12    const types = [  
13      this.surveyTypes.coursal,  
14      this.surveyTypes.nonCoursal,  
15      this.surveyTypes.both,  
16    ];  
17    if (this.isFacultyFA) {  
18      types.push(this.surveyTypes.atelier);  
19    }  
20    return types;  
21  },  
22 }
```

■ Výpis kódu 4.6 Nové funkcie v sekcii `computed`

V dôsledku zmeny API backendu, ktorá vyžadovala jednoznačné rozlíšenie medzi jednotlivými typmi ankiet, bolo nevyhnutné upraviť aj logiku frontendovej funkcie zodpovednej za odosielanie týchto údajov na server. Táto úprava spočívala v modifikácii tela HTTP požiadavky.

```
1 body: JSON.stringify({  
2   facultyId: this.faculty.orgUnitCode,  
3   semesterCode: this.semester,  
4   coursals,  
5   atelier // New field  
6 },
```

■ Výpis kódu 4.7 Pridaný atribút do tela HTTP požiadavky

4.2 Správa privilegovaných používateľov

4.2.1 Úpravy na backende

Ako bolo uvedené v sekcii 3.2.2, implementácia tejto požiadavky si vyžiadala vytvorenie viacerých tried a rozhraní s cieľom zachovať navrhnutú architektúru systému. V prvom kroku bola vytvorená entita `AllowedTeacher`, ktorá je namapovaná na tabuľku `RES_ALLOWED_TEACHERS_PRIV`. V uvedenej tabuľke je na identifikáciu jednotlivých záznamov použitý zložený primárny kľúč pozostávajúci z atribútov `ID_TEACHER`, `ID_FACULTY`, `ROLE` a `DEPT`. Dôvodom takejto implementácie je skutočnosť, že primárny kľúč definovaný iba pomocou jedného atribútu by bol nedostatočný, keďže mnohí učitelia majú v tejto tabuľke viacero záznamov, a to aj v rámci jednej fakulty.

```

1  @Entity
2  @Table(name = "RES_ALLOWED_TEACHERS_PRIV", schema = "NOVA_ANKETA")
3  @IdClass(AllowedTeacherPK.class)
4  class AllowedTeacher(
5      @field:Column(name = ID_TEACHER_COLUMN)
6      @field:Id
7      val idTeacher: Long,
8      @field:Column(name = ID_FACULTY_COLUMN)
9      @field:Id
10     val idFaculty: Long,
11     ...
12 )

```

■ Výpis kódu 4.8 Ústrižok z entity `AllowedTeacher`

Pre prácu s týmito entitami a databázou bolo taktiež implementované rozhranie `AllowedTeacherRepository`, do ktorého pre vypracovanie tejto požiadavky nebolo nič potrebné pridávať, nakoľko Spring Data JPA automaticky poskytuje implementáciu štandardných CRUD operácií na prácu s databázou.

```

1  @Repository
2  interface AllowedTeacherRepository : JpaRepository<AllowedTeacher,
    ↳ AllowedTeacherPK>

```

■ Výpis kódu 4.9 Rozhranie `AllowedTeacherRepository`

Ako ďalšia bola implementovaná trieda `AllowedTeacherMapper`, ktorá zabezpečuje obojsmernú konverziu medzi DTO objektmi a entitami doménového modelu. Pri jej implementácii bola zachovaná konzistentná štruktúra a architektonický vzor, ktorý vychádza z existujúcich mapovacích tried v rámci systému. Samotná DTO trieda nebola explicitne implementovaná, keďže, ako už

bolo uvedené v sekcii 2.3.4, tento projekt generuje triedy na základe YAML špecifikácie, v ktorej sú definované aj DTO triedy. Z tohto dôvodu bolo nevyhnutné príslušnú YAML špecifikáciu rozšíriť a štruktúru nového DTO objektu v nej explicitne zadať.

```
1 @Component
2 class AllowedTeacherMapper {
3     fun toDto(entity: AllowedTeacher): AllowedTeacherDto =
4         ↪ entity.toDtoImpl()
5     fun toEntity(dto: AllowedTeacherCreateDto): AllowedTeacher =
6         ↪ dto.toEntityImpl()
7     fun AllowedTeacher.toDtoImpl(): AllowedTeacherDto =
8         ↪ AllowedTeacherDto(
9             ...
10        )
11     fun AllowedTeacherCreateDto.toEntityImpl(): AllowedTeacher =
12         ↪ AllowedTeacher(
13             ...
14        )
15 }
```

■ Výpis kódu 4.10 Trieda AllowedTeacherMapper

Následne bola implementovaná servisná vrstva, do ktorej boli postupne dopĺňané jednotlivé CRUD operácie. V počiatočnej fáze bola z dôvodu zjednodušenia testovania funkcionality realizovaná iba metóda na získanie dát. V záujme dodržania projektových konvencií bolo najskôr vytvorené rozhranie, ktoré definuje hlavičky jednotlivých operácií, a až následne jeho konkrétna implementácia vo forme triedy obsahujúcej samotnú aplikačnú logiku.

```
1 @Service
2 class IAllowedTeacherServiceImpl(
3     private val allowedTeacherRepository: AllowedTeacherRepository,
4     private val allowedTeacherMapper: AllowedTeacherMapper
5 ) : IAllowedTeacherService {
6     @PreAuthorize("hasAuthority('SUPER_ADMIN')")
7     override fun getAllowedTeachers(): List<AllowedTeacherDto> {
8         return allowedTeacherRepository.findAll().map {
9             ↪ allowedTeacherMapper.toDto(it) }
10    }
11 }
```

■ Výpis kódu 4.11 Prvotná implementácia konkrétnej triedy AllowedTeacherService

Na spracovanie HTTP požiadaviek bola implementovaná anotovaná trieda `@RestController`, ktorá je povinná implementovať metódy definované v YAML

špecifikácii. Generovaním abstraktných tried z YAML špecifikácie vznikla trieda `AllowedTeachersApi`, z ktorej dedí trieda `AllowedTeacherController`. Hlavnou výhodou tohto prístupu je, že mechanizmus dedičnosti striktné vynucuje implementáciu všetkých koncových bodov (angl. *endpoints*) definovaných v API, čím sa minimalizuje riziko nekonzistencií medzi špecifikáciou a jej konkrétnou implementáciou.

```
1 @RestController
2 class AllowedTeacherController(
3     private val allowedTeacherService: IAllowedTeacherService,
4 ) : AllowedTeachersApi {
5     override fun getAllowedTeachers():
6         ↳ ResponseEntity<List<AllowedTeacherDto>> =
7             ok(allowedTeacherService.getAllowedTeachers())
8 }
```

■ Výpis kódu 4.12 Prvotná implementácia triedy `AllowedTeacherController`

Postupne bola doplnená logika na vytváranie nových záznamov, odstraňovanie existujúcich záznamov a ich aktualizáciu do vrstiev `Controller`, `Service` a do špecifikácie API.

4.2.2 Úpravy na frontende

Na úrovni frontendu bolo potrebné vytvoriť novú stránku, na ktorej budú privilegované roly zobrazené v tabuľke rozdelenej na dve časti – *automat* a *manuál*. Všetky zobrazované údaje sa získavajú prostredníctvom volania backendových rozhraní, pričom backend čerpá dáta z databázy.

S cieľom sprístupniť túto stránku používateľovi bol aktualizovaný komponent `Sidenav.vue`, ktorý zabezpečuje zobrazenie zoznamu dostupných stránok v bočnom navigačnom paneli na ľavej strane používateľského rozhrania. Následne bola implementovaná nová stránka `PrivilegedRoles.vue`.

Stránka `PrivilegedRoles.vue` využíva komponent `NuxtLayout`, ktorá zabezpečuje vykreslenie stránky v rámci predvoleného rozloženia aplikácie definovaného v `layouts/default.vue`. Toto základné rozloženie obsahuje hornú navigačnú lištu, bočné navigačné menu a hlavný obsahový priestor, do ktorého sa vkladá obsah konkrétnych podstránok. Tým je zaručené, že novovytvorená stránka automaticky preberá jednotný vizuálny rámec a používateľský štýl zvyšku aplikácie.

4.3 Nový typ otázok

4.3.1 Úpravy na backende

Implementácia mechanizmov na spracovanie odpovedí na otázky, ako aj modelovanie vzťahov medzi jednotlivými otázkami a ich odpovedovými možnosťami, musela byť realizovaná s ohľadom na existujúci návrh databázovej schémy. Jednotlivé odpovedové možnosti sa ukladajú v tabuľke `QUESTION_OPTION`, čo si vyžiadalo návrh a implementáciu novej zodpovedajúcej entity v dátovom modeli.

Následne bola implementovaná entita namapovaná na databázovú tabuľku `QUESTION_OPTION_SET`, ktorá zabezpečuje logické zoskupovanie odpovedových možností do sád. Poslednou potrebnou entitou bola entita mapujúca vzťah medzi sadami a jednotlivými možnosťami, reprezentovaný databázovou tabuľkou `QUESTION_OPTION_ORDER_SET`, v ktorej jeden záznam obsahuje identifikátor sady, identifikátor možnosti a atribút určujúci poradie danej možnosti v rámci príslušnej sady. Pre tieto nové entity bolo potrebné podobne ako v predchádzajúcej požiadavke s privilegovanými rolami vytvoriť repozitárové rozhrania a aktualizovať logiku servisnej vrstvy.

```
1 @Entity
2 @Table(name = "QUESTION_OPTION", schema = "NOVA_ANKETA")
3 class QuestionOption(
4     @field:Column(name = QUESTION_OPTION_KEY_COLUMN)
5     @field:Id
6     val questionOptionKey: Long?,
7     @field:Column(name = OPTION_TEXT_CZ_COLUMN)
8     val optionTextCz: String?,
9     @field:Column(name = OPTION_TEXT_EN_COLUMN)
10    val optionTextEn: String?,
11 )
```

■ Výpis kódu 4.13 Entita reprezentujúca odpovedovú možnosť

V rámci tímového softvérového projektu boli do tabuľky `QUESTION` pridané nové stĺpce, ktoré explicitne špecifikujú, či je daná otázka viacvýberového typu, či umožňuje zvoliť viacero odpovedí súčasne a či podporuje zadanie vlastnej odpovede. Táto zmena si vyžiadala rozšírenie existujúcej entity reprezentujúcej otázky o nové typy otázok, ako aj úpravu príslušnej mapovacej triedy, aby uvedené nové logické atribúty boli korektne zohľadnené a perzistentne reprezentované v dátovom modeli. Taktiež bolo nevyhnutné na úrovni entity zadať vzťah medzi otázkou a príslušnou sadou odpovedových možností.

```

1 class Question(
2     ...
3     @field:Column(name = IS_MULTIANSWER_OPTION_COLUMN, nullable =
4         ↳ false)
5     val isMultiAnswerOption: Boolean,
6     @field:Column(name = HAS_MULTICHOICE_ANSWER_COLUMN, nullable =
7         ↳ false)
8     val hasMultichoiceAnswer: Boolean,
9     @field:Column(name = HAS_CUSTOM_OPTION_COLUMN, nullable = false)
10    val hasCustomOption: Boolean,
11    @field:OneToMany(mappedBy = "question", fetch = FetchType.LAZY)
12    @field:OrderBy("questionOptionSetKey ASC")
13    val optionSets: List<QuestionOptionSet> = emptyList()
14 )

```

■ **Výpis kódu 4.14** Nové atribúty v entite Question

4.3.2 Úpravy na frontende

Na strane frontendu boli na základe návrhu riešenia uvedeného v sekcii 3.3.2 vykonané úpravy, ktoré sa primárne týkali komponentu `AddQuestionModal.vue`. Bola doplnená funkcionálna na vytváranie otázok typu *výber jednej možnosti* a *výber viacerých možností*. Pri zvolení niektorého z týchto typov otázok sa v spodnej časti modálneho okna zobrazí rozhranie umožňujúce vytvoriť jednotlivé odpovedňové možnosti, definovať ich anglické ekvivalenty, meniť ich poradie a taktiež špecifikovať názov sady týchto možností.

■ **Obr. 4.1** Zmeny v modálnom okne pri vytváraní otázky

4.4 Refaktorizácia a dokumentácia PL/SQL kódu

4.4.1 MERGE_ALL_UMAP_ROLES

Procedúra `MERGE_ALL_UMAP_ROLES` je spúšťaná na dennej báze prostredníctvom plánovanej úlohy `ANK_ANKETA.MERGE_ALL_UMAP_ROLES`. Táto procedúra zabezpečuje spracovanie všetkých záznamov o roliach používateľov z databázovej tabuľky `ANK_ANKETA.MV_USERMAP_ROLE_ANKETA`. Z každého záznamu extrahuje technickú rolu v Usermapu a kód fakulty, ktoré následne mapuje na číselné identifikátory organizačných jednotiek. V prípade, že je príslušná fakulta identifikovateľná, procedúra v závislosti od typu technickej roly volá pomocnú procedúru `MERGE_UMAP_ROLE`, ktorá s danými údajmi vykonáva ďalšie spracovanie. V rámci tejto PL/SQL procedúry sú rozpoznávané nasledujúce technické role v Usermap:

Správca aplikácie

Mapovanie: Do `MERGE_UMAP_ROLE` sa posiela rola `dept` a `rep`.

Práva: Celková správa nastavení subsystému Anketa, príprava behov ankiet, ich vyhodnocovanie, realizácia cenzúry a primárne nastavovanie rolí používateľov.

Fakultný správca

Mapovanie: Do `MERGE_UMAP_ROLE` sa posiela rola `dept` a `rep`.

Práva: Správa behu fakultných ankiet, nastavovanie harmonogramu, otváranie/uzatváranie ankiet, správa fakultných otázok a kontrola predmetov či vyučujúcich.

Vedenie

Mapovanie: Do `MERGE_UMAP_ROLE` sa posiela rola `dept` a `rep`.

Práva: Vo fáze prípravy uzavretej ankety na zverejnenie má prístup a vidí všetky komentáre študentov ku všetkým vyučujúcim.

Komentátor

Mapovanie: Do `MERGE_UMAP_ROLE` sa posiela rola `rep`.

Práva: Vo fáze prípravy uzavretej ankety má oprávnenie vyjadrovať sa k študentským komentárom v nepredmetovej ankete.

Prvým krokom refaktorizácie bolo zlepšenie spracovania okrajových a chybových prípadov využitím už existujúceho mechanizmu logovania. Analýza procedúr a tabuliek v schéme `NOVA_ANKETA` preukázala, že takýto mechanizmus je implementovaný vo forme procedúry `PROCEDURE_LOG` a tabuľky `LOG_TABLE`.

Z hľadiska dlhodobej udržateľnosti a rozšíriteľnosti kódu bolo žiadúce zaviesť centralizovanú lookup tabuľku. Tým sa eliminovala potreba manuálne definovať konštanty v jednotlivých procedúrach a zároveň sa pri zmene konkrétnej hodnoty odstránila nutnosť upravovať viacero procedúr, v ktorých je

táto hodnota použitá. Krátka analýza existujúcich procedúr a funkcií identifikovala viaceré objekty, ktoré by z takejto úpravy profitovali, napríklad funkcia `TEST_VYPLNENA_ANKETA_JSON` alebo procedúra `MERGE_UMAP_ROLE`.

V súlade s odporúčaniami spoločnosti Oracle bol dátový typ `VARCHAR` nahradený dátovým typom `VARCHAR2`, ktorý je v prostredí Oracle Database štandardom pre ukladanie reťazcových údajov a poskytuje garantovanú stabilitu správania pri spracovaní znakových reťazcov aj v budúcich verziách databázového systému [26].

4.4.2 `MERGE_UMAP_ROLE`

Procedúra `MERGE_UMAP_ROLE` zabezpečuje vloženie alebo aktualizáciu roly používateľa v tabuľke `RES_ALLOWED_TEACHERS_PRIV`. V prípade, že parameter `p_org_unit = 0`, čo znamená, že operácia sa vzťahuje na celú univerzitu ČVUT, rola sa nastaví na všetkých fakultách. V takomto prípade sa procedúra rekurzívne volá pre každú fakultu zo zoznamu organizačných jednotiek.

Ak je špecifikovaná konkrétna fakulta, najskôr sa overí, či už používateľ danú rolu v tabuľke má. V kladnom prípade sa vykoná operácia `UPDATE` s nastavením aktuálnych metaúdajov. V zápornom prípade sa vykoná operácia `INSERT`. Pri nej sa najprv pokúsi nájsť `ID_TEACHER` v pohľade `MV_TEACHER` a ak sa ho nepodari nájsť, použije sa vstupné identifikátorové pole `ID`. Výsledkom je upsertová logika, ktorá udržiava tabuľku `RES_ALLOWED_TEACHERS_PRIV` v súlade so stavom evidovaným v systéme Usermap.

Procedúra je volaná nadradenou procedúrou `MERGE_ALL_UMAP_ROLES` v danom intervale. Analogicky ako pri `MERGE_ALL_UMAP_ROLES`, aj v tomto prípade bolo na definovanie konštánt zavedené využitie pomocnej lookup tabuľky namiesto ich pevného zakódovania priamo do zdrojového kódu.

Aktuálna implementácia využívala postup `SELECT COUNT(*)` a následný `UPDATE` alebo `INSERT`, čím vznikol priestor na vznik súbehu (angl. *race condition*), čo znamená, že medzi oboma krokmi mohla paralelná transakcia zmeniť dáta a viesť k inkonzistentnému stavu alebo zlyhaniu operácie. Preto bol tento postup nahradený príkazom `MERGE`, ktorý integruje rozhodovaciu a modifikačnú logiku do jednej atomickej databázovej operácie. Tým sa eliminuje riziko kolízií medzi transakciami, zvyšuje sa výkon a zároveň sa zlepšuje čitateľnosť aj udržiavateľnosť kódu.

```
1 CREATE OR REPLACE PROCEDURE merge_umap_role (  
2     p_uname VARCHAR2, -- VARCHAR2 instead of VARCHAR  
3     ...  
4 ) AS  
5     ...  
6 BEGIN  
7     -- Lookup full name from usermap  
8     l_full_name := ank_anketa.f_usermap_full_name(p_uname);  
9  
10    -- Handle university-level role (apply to all faculties)  
11    IF p_org_unit = 0 THEN  
12        -- Log university-level role assignment  
13        PROCEDURE_LOG(  
14            'merge_umap_role_university_level',  
15            'Assigning role ' || p_role || ' to user ' || p_uname || '  
16            ↳ for all faculties'  
17        );  
18    ELSE  
19        ...  
20        -- Use MERGE statement for atomic upsert operation  
21        MERGE INTO res_allowed_teachers_priv target  
22        ...  
23    END IF;  
24 END merge_umap_role;  
25 /
```

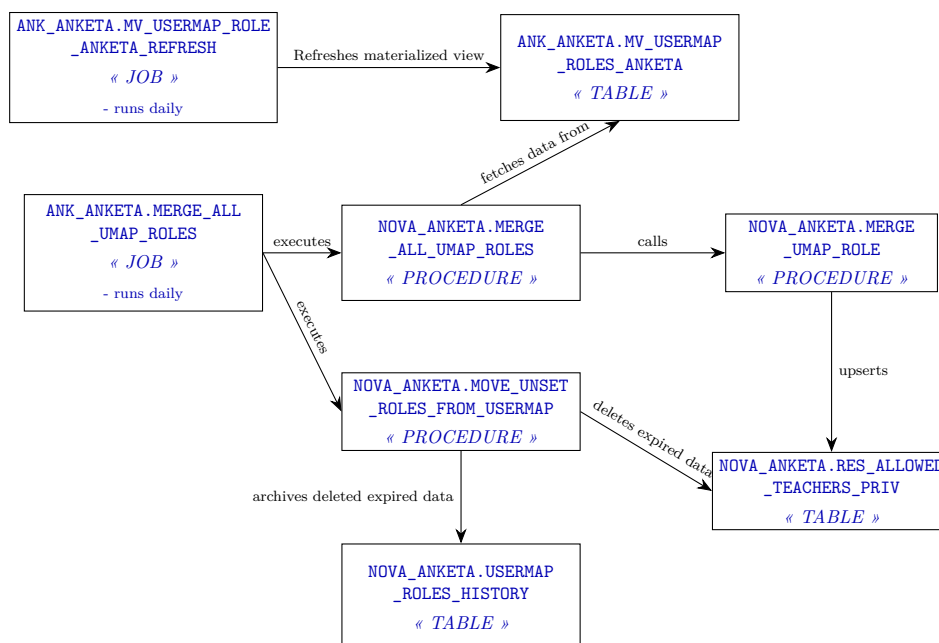
■ Výpis kódu 4.15 Ústrižok z implementovaných zmien v MERGE_UMAP_ROLE

4.4.3 MOVE_UNSET_ROLES_FROM_USERMAP

Procedúra `MOVE_UNSET_ROLES_FROM_USERMAP` zabezpečuje presun expirovaných rolí z databázovej tabuľky `RES_ALLOWED_TEACHERS_PRIV` do archívnej tabuľky `USERMAP_ROLES_HISTORY`. Operácia sa týka výlučne tých záznamov, ktoré boli do systému vložené automatizovaným mechanizmom, teda majú hodnotu `CHANGER = 'AUTOMAT'`) a pre ktoré je hodnota atribútu `CHANGED` staršia ako dátum špecifikovaný parametrom `P_WHEN`.

Parameter `P_WHEN` je konfigurovateľný a umožňuje definovať ľubovoľný hraničný dátum. Ak nie je explicitne zadaný, používa sa predvolená hodnota `SYSDATE - 6`, čo znamená, že procedúra spracováva záznamy staršie ako 6 dní. Po presune sú tieto historické roly z tabuľky `RES_ALLOWED_TEACHERS_PRIV` odstránené, aby táto tabuľka obsahovala výhradne aktuálne údaje.

Úloha `ANK_ANKETA.MERGE_ALL_UMAP_ROLES` spúšťa túto procedúru. Ide o krátku procedúru s minimálnym priestorom na refaktORIZáciu a preto boli možné iba drobné stylistické úpravy.



■ Obr. 4.2 Diagram popisujúci dané PL/SQL procedúry

4.5 Dokumentácia Oracle Scheduler úloh na úrovni databázy

Oracle Scheduler úlohy obsahujú v Oracle SQL Developer množstvo detailných atribútov, ako sú dátum vytvorenia, opakovací interval, majiteľ úlohy a ďalšie konfiguračné parametre. Pre potreby tvorby technickej dokumentácie sú však kľúčové najmä nasledujúce údaje: názov úlohy, jej frekvencia spúšťania, hodnota poľa *enable* indikujúca, či je úloha aktuálne aktívna, a samotný kód Oracle Scheduler úlohy. Tieto informácie boli spísané do Markdown dokumentu a odovzdané vedúcemu bakalárskej práce prostredníctvom GitLab wiki stránky.

V rámci schémy ANK_ANKETA boli identifikované nasledujúce Oracle Scheduler úlohy definované:

■ MERGE_ALL_UMAP_ROLES

Atribút	Hodnota
Frekvencia	Denne o 10:45
Enabled	TRUE

Spúšťa procedúry: MERGE_ALL_UMAP_ROLES a MOVE_UNSET_ROLES_FROM_USERMAP. Zabezpečuje celý proces synchronizácie používateľských rolí z usermapy do systému Anketa.

```
begin
    nova_anketa.merge_all_umap_roles;
    nova_anketa.move_unset_roles_from_usermap();
end;
```

■ MV_PARALLEL_REFRESH

Atribút	Hodnota
Frekvencia	Denne o 01:47
Enabled	TRUE

Obnovuje materializovaný pohľad MV_PARALLEL. Ostatné pohľady (MV_TEACHER, MV_STUDENT, MV_TEACHING_TYPE, MV_STUDENT_ENROLLED_COURSE) sú v kóde zakomentované a týmto jobom sa neobnovujú.

```
BEGIN
    DBMS_MVIEW.REFRESH('MV_PARALLEL','C');
END;
```

■ MV_STUDENT_ENROLLED_COURSE_RF

Atribút	Hodnota
Frekvencia	Denne o 09:30
Enabled	TRUE

Obnovuje materializovaný pohľad MV_STUDENT_ENROLLED_COURSE. Spúšťa sa 30 minút po MV_PARALLEL_REFRESH, čím sa zaručuje správne poradie obnovenia.

```
BEGIN
    DBMS_MVIEW.REFRESH('MV_STUDENT_ENROLLED_COURSE','C');
END;
```

■ MV_TEACHING_TYPE_REFRESH

Atribút	Hodnota
Frekvencia	Denne o 09:15
Enabled	TRUE

Obnovuje materializovaný pohľad MV_TEACHING_TYPE. Spúšťa sa 15 minút po MV_PARALLEL_REFRESH.

```
BEGIN
    DBMS_MVIEW.REFRESH('MV_TEACHING_TYPE','C');
END;
```

■ MV_USERMAP_ROLE_ANKETA_REFRESH

Atribút	Hodnota
Frekvencia	Denne
Enabled	TRUE

Obnovuje materializovaný pohľad MV_USERMAP_ROLE_ANKETA, ktorý obsahuje aktuálne dáta o roliach používateľov z usermapy. Tento pohľad je zdrojom dát pre procedúru MERGE_ALL_UMAP_ROLES.

```
begin
    DBMS_MVIEW.REFRESH('MV_USERMAP_ROLE_ANKETA','C');
end;
```

■ SNAPSHOTS_REFRESH

Atribút	Hodnota
Frekvencia	Denne o 09:00
Enabled	FALSE

Momentálne je vypnutý. Podľa kódu by refreshoval pohľady MV_PARALLEL, MV_TEACHING_TYPE a MV_STUDENT_ENROLLED_COURSE v jednom bloku s commitmi medzi nimi.

```
BEGIN
    DBMS_MVIEW.REFRESH('MV_PARALLEL','C');
commit;
```



```
DBMS_MVIEW.REFRESH('MV_TEACHING_TYPE','C');
commit;
DBMS_MVIEW.REFRESH('MV_STUDENT_ENROLLED_COURSE','C');
END;
```

■ TPREDMT_SET_NULL

Atribút	Hodnota
Frekvencia	Každú minútu
Enabled	TRUE

Nastavuje `is_excluded = NULL` pre záznamy v tabuľke `t_course_in_anketa`, kde `is_excluded = 'N'`. Beží každú minútu.

```
begin
  update t_course_in_anketa
  set is_excluded = null
  where is_excluded = 'N';
commit;
end;
```

4.6 Úprava rozhrania pre správu používateľov

Na základe navrhovaného riešenia bola aktualizovaná špecifikácia API pre používateľov tak, aby obsahovala aj atribút pre poznámku typu `string`. Súčasne bolo nevyhnutné upraviť mapovaciu triedu tak, aby na frontend odosielať všetky požadované polia. Úpravy na frontende si vyžiadali zásah do komponentu `AddUserModal`, ktorý reprezentuje modálne okno na vytváranie nových používateľov, resp. správcov, s cieľom doplniť možnosť vyplnenia textového poľa na zadanie poznámky. Okrem toho boli implementované zmeny umožňujúce vyhľadávanie a filtrovanie v zozname používateľov.

Uživatelia

Q_vyhledat

Id ↑	Fakulta	Užívateľské meno	Jméno a příjmení	Role	Poznámka	Akce
11	Fakulta biomedicínského inženýrství	HONZDE	Zdenek Hon	admin		✎ 🗑
12	Fakulta biomedicínského inženýrství	PETRUJOL	Jonás Petrik	admin		✎ 🗑
13	Fakulta informačních technologií	LIBANALE	Alena Libánská	admin		✎ 🗑
14		KUBINMIL 🟡	Milan Kubín	superadmin		✎ 🗑
15	Fakulta architektury	HEJLEVOJ	Vojtěch Hejsek	admin	Vývoj FA atelierů	✎ 🗑
16	Masarykův ústav	WOSTRALZ	Alžběta Wostřá	admin		✎ 🗑
17	Fakulta informačních technologií	BERKEADA	Adam Berkeš	admin	Vývoj BE Admin	✎ 🗑
18	Fakulta informačních technologií	LIPTAROB	Robert Lipták	admin	Admin - správa odpovědí fakultní ankety	✎ 🗑
19	Fakulta informačních technologií	DURKOJAK 🟡	Jakub Durkovič	superadmin	Vývoj BE+FE Admin	✎ 🗑
20	Fakulta informačních technologií	HELLEJIR	Jiri Heller	admin	Vývoj FE	✎ 🗑

Items per page: 10 11:20 of 22 < > >1

■ Obr. 4.3 Stránka pre správu administrátorov

4.7 Úprava otázok a zoznamov otázok

Na základe analýzy aktuálnej implementácie backendovej logiky bolo zistené, že už existuje mechanizmus, ktorý zabráňuje vymazaniu otázok patriacich do existujúcich sád otázok, ako aj vymazaniu zoznamov otázok, ktoré sú už priradené k anketám. Uvedený mechanizmus je implementovaný v repozitárových rozhraniach prislúchajúcich k daným entitám prostredníctvom metódy `isDeletable`. Z tohto dôvodu implementácia predmetnej požiadavky vyžaduje zmeny iba na úrovni frontendu.

V prípade pokusu o takéto nedovolené mazanie backend vracia chybový kód HTTP 400. Na frontende bolo po úprave implementované zachytávanie tohto chybového kódu a zobrazenie informačnej správy používateľovi, ktorá ho notifikuje o neúspešnom vymazaní otázky alebo zoznamu otázok z dôvodu ich väzby na inú sadu otázok alebo existujúcu anketu.

Počas testovania tejto funkcionality bol zároveň identifikovaný nový, pôvodne nezdokumentovaný chybový stav súvisiaci s chýbajúcou českou alebo anglickou jazykovou verziou otázky. V situácii, keď používateľ nevyplnil jednu z jazykových verzií, backend vracal chybový kód HTTP 500. Na odstránenie tohto problému bola na úrovni frontendu zavedená validácia používateľského vstupu, ktorá pri absencii českej alebo anglickej verzie zablokuje odoslanie požiadavky a vyzve používateľa na doplnenie chýbajúcej verzie.

[illegible]

5.1 Manuálne testovanie

V porovnaní s automatizovanými testami je tento prístup náročnejší z hľadiska požadovaného úsilia a zároveň vykazuje vyššiu náchylnosť na zlyhanie ľudského faktora [27]. Na manuálnom testovaní sa podieľal autor práce, jej vedúci a administrátori anketového systému.

5.2 Jednotkové testovanie

37

Z hľadiska štruktúry jednotkové testy spravidla sledujú vzor AAA:

- **Arrange** (príprava): inicializácia testovacích dát a závislostí,
- **Act** (vykonanie): spustenie testovaného kódu,
- **Assert** (overenie): kontrola, či výsledok zodpovedá očakávanému správaniu.

Kľúčovým princípom unit testovania je izolácia testovanej jednotky. Unit test nesmie byť závislý od externých systémov, ako sú databáza, súborový systém alebo sieťová infraštruktúra. Na dosiahnutie tejto izolácie sa používajú test doubles, napríklad mocky alebo stuby, ktoré substituujú skutočné závislosti kontrolovanými a predvídateľnými implementáciami [28].

V rámci vypracovania požiadavky na správu privilegovaných používateľov bol vytvorený jednotkový test pre `AllowedTeacherMapper`, ktorý testuje správnosť konverzie medzi DTO a entitami. Na testovanie sa využíva framework *Kotest*, čo je Kotlin alternatívou k *JUnit*.

```

1 class AllowedTeacherMapperTest(
2     private val mapper: AllowedTeacherMapper,
3 ) : ShouldSpec({
4     val entity = AllowedTeacher(1L, 18L, "GUARANTOR", "K13136",
5         ↪ "username", "Full Name", "manual")
6     val expectedDto = AllowedTeacherDto(1L, 18L, "GUARANTOR",
7         ↪ "K13136", "username", "Full Name", "manual")
8     val createDto = AllowedTeacherCreateDto(1L, 18L, "GUARANTOR",
9         ↪ "K13136", "username", "Full Name")
10
11     should("map entity to dto") {
12         val result = mapper.toDto(entity)
13         result shouldBe expectedDto
14     }
15
16     should("map create dto to entity") {
17         val result = mapper.toEntity(createDto)
18         result.idTeacher shouldBe createDto.idTeacher
19         ...
20     }
21 })

```

■ Výpis kódu 5.1 Jednotkový test na skontrolovanie získania všetkých učiteľov

Test *retrieve all allowed teachers* overuje, že metóda `getAllowedTeachers` korektne vracia zoznam všetkých povolených učiteľov. V časti Arrange je repositár nakonfigurovaný tak, aby volanie metódy `findAll` vrátilo zoznam obsahujúci jeden objekt reprezentujúci učiteľa. V časti Act je následne vykonané volanie testovanej metódy. V časti Assert sa verifikuje, že výsledný zoznam

obsahuje práve jeden prvok, čím sa potvrdzuje korektná funkcionálna metóda pri spracovaní dostupných údajov z repozitára.

5.3 Integračné testovanie

Integračné testovanie predstavuje metodologický prístup v oblasti testovania softvéru, pri ktorom sa jednotlivé komponenty alebo moduly aplikácie systematicky spájajú a overujú s cieľom posúdiť kvalitu a korektnosť ich vzájomnej spolupráce. Jeho primárnou úlohou je verifikovať, že integrované časti systému sú schopné navzájom konzistentne komunikovať, správne si odovzdávať údaje a úspešne realizovať požadované interakcie v súlade so špecifikovanými funkčnými a nefunkčnými požiadavkami [29].

V rámci správy privilegovaných používateľov bol vytvorený integračný test na overenie funkčnosti príslušnej triedy typu **Controller**. Prostredníctvom anotácie **@WebMvcTest** sa konfiguruje a spúšťa webová vrstva aplikácie, pričom sa na simuláciu HTTP požiadaviek využíva nástroj **MockMvc**. Závislá vrstva **AllowedTeacherService** je v tomto prípade simulovaná pomocou knižnice *MockK*. Nejde teda o volanie reálnej biznis logiky, ale o jej programovú náhradu, ktorá vracaním preddefinovaných hodnôt umožňuje izolované testovanie správania sa webového rozhrania.

Na meranie pokrytia kódu testami bol využitý nástroj JaCoCo, integrovaný priamo do Gradle zostavovacieho procesu. Trieda **AllowedTeacherController** dosiahla pokrytie 100%, čo potvrdzuje, že všetky vetvy a inštrukcie kontroléra boli pokryté definovanými testovacími prípadmi.

```
1 should("retrieve all allowed teachers") {
2     every { allowedTeacherService.getAllowedTeachers() } returns
3         listOf(allowedTeacherDto)
4
5     val expectedJson = TestObjectMapper.mapper
6         .writeValueAsString(listOf(allowedTeacherDto))
7
8     mvc.perform(
9         MockMvcRequestBuilders.get(path)
10    )
11    .andExpect(MockMvcResultMatchers.status().is2xxSuccessful)
12    .andExpect(MockMvcResultMatchers.content().json(expectedJson))
13 }
```

■ **Výpis kódu 5.2** Integračný test kontroluje správne namapovanie GET požiadavku

Kapitola 6

Záver

Cieľom tejto bakalárskej práce bolo rozšírenie administrátorského rozhrania aplikácie Anketa ČVUT a odstránenie vybraných známych chýb. Jednotlivé požiadavky na úpravy boli systematicky analyzované, pre každú z nich bolo navrhnuté riešenie, ktoré bolo následne implementované, primerane zdokumentované a podrobené testovaniu.

Výstupy práce majú za cieľ pozitívne ovplyvniť činnosť administrátorov ankety, vrátane superadministrátora. Implementované zmeny prispievajú k zefektívneniu procesu vytvárania ateliérovej ankety prostredníctvom administrátorského rozhrania, zjednodušujú správu privilegovaných rolí pre superadministrátora a rozširujú možnosti získavania spätnej väzby o otázky s viacerými odpoveďami. Dodaná dokumentácia k PL/SQL skriptom a Oracle Scheduler úlohám zvyšuje prehľadnosť a uľahčuje prácu s databázovou vrstvou, ako aj s procesmi bežiacimi na pozadí. Táto dokumentácia zároveň predstavuje vhodný východiskový základ pre tvorbu budúcej dokumentácie. Implementované vizuálne úpravy na frontende zároveň prispievajú k zlepšeniu používateľskej skúsenosti.

Realizované riešenia otvárajú priestor na ďalší rozvoj administrátorského rozhrania, napríklad v oblasti tvorby ateliérovej ankety, kde by bolo možné zaviesť priame nahrávanie mapovacej matice prostredníctvom používateľského rozhrania. Na základe diskusie s vedúcim práce možno konštatovať, že zadanie bolo splnené a implementované v súlade s pôvodnými požiadavkami a očakávaniami.

Bibliografia

1. BERKEŠ, Adam. *Anketa ČVUT – Admin rozhraní – aktualizace a rozšíření backend* [online]. Praha, 2025 [cit. 2026-03-06]. Dostupné z : <http://dspace.cvut.cz/handle/10467/118318>. Bakalárska práca. České vysoké učení technické v Praze, Fakulta informačních technologií.
2. HEJLEK, Vojtěch. *Anketa ČVUT – návrh a realizace atelierové ankety pro Fakultu architektury* [online]. Praha, 2025 [cit. 2026-03-06]. Dostupné z : <https://dspace.cvut.cz/handle/10467/118337>. Bakalárska práca. České vysoké učení technické v Praze, Fakulta informačních technologií.
3. LINNIK, Irina. *Web Application Architecture: the main types and models* [online]. 2023. [cit. 2026-03-06]. Dostupné z : <https://softteco.com/blog/web-application-architecture-explained>.
4. HASSAN, Mahnoor. *Persistent Layer Design for Web Apps* [online]. 2026. [cit. 2026-03-06]. Dostupné z : <https://www.scribd.com/document/974724143/Lecture-19-Persistence-Layer>.
5. DEVRT. *All about Oracle Database - Definition, Features, Benefits* [online]. 2023. [cit. 2026-03-06]. Dostupné z : <https://www.devart.com/dbforge/oracle/all-about-oracle-database/>.
6. AWATI, Rahul; MOORE, Lindsay. *What is PL/SQL (Procedural Language/Structured Query Language)?* [online]. 2025. [cit. 2026-03-06]. Dostupné z : <https://www.techtarget.com/searchoracle/definition/PL/SQL>.
7. STELITANO, Luiza. *What Is Backend? A Comprehensive Intro to Server-Side Development* [online]. 2025. [cit. 2026-03-06]. Dostupné z : <https://alokai.com/blog/what-is-backend>.
8. CONSTRUCTOR UNIVERSITY. *Kotlin: A modern and expressive programming language for everyone* [online]. 2023. [cit. 2026-03-06]. Dostupné z : <https://constructor.university/blog/kotlin-programming-language>.

9. IBM. *What Is Java Spring Boot?* [online]. 2024. [cit. 2026-03-06]. Dostupné z : <https://www.ibm.com/think/topics/java-spring-boot>.
10. ALTVATER, Alexandra. *What is N-Tier Architecture?* [online]. 2017. [cit. 2026-03-06]. Dostupné z : <https://stackify.com/n-tier-architecture/>.
11. ORACLE. *Entities – The Java EE 6 Tutorial* [online]. 2013. [cit. 2026-03-06]. Dostupné z : <https://docs.oracle.com/javaee/6/tutorial/doc/bnbqa.html>.
12. ORACLE. *Technical Architecture* [online]. 2017. [cit. 2026-03-06]. Dostupné z : https://docs.oracle.com/cd/E76310_01/pdf/141/html/operations_guide/reim-og-architecture.htm.
13. HÜRIYETOĞLU, Merve. *Spring Boot Layered Architecture: Presentation Layer* [online]. 2025. [cit. 2026-03-06]. Dostupné z : <https://medium.com/javarevisited/spring-boot-layered-architecture-presentation-layer-controller-7fa6e28540a7>.
14. LOPES, Pedro. *API First Development with Spring Boot and OpenAPI 3.0 / Baeldung* [online]. 2023. [cit. 2026-03-06]. Dostupné z : <https://www.baeldung.com/spring-boot-openapi-api-first-development>.
15. CLOUDINARY. *Front-End Development: The Complete Guide* [online]. 2026. [cit. 2026-03-07]. Dostupné z : <https://cloudinary.com/guides/front-end-development/front-end-development-the-complete-guide>.
16. MARICHEVA, Anna. *History of JavaScript* [online]. 2023. [cit. 2026-03-06]. Dostupné z : <https://softteco.com/blog/history-of-javascript>.
17. SANITY. *Vue.js overview* [online]. 2024. [cit. 2026-03-06]. Dostupné z : <https://www.sanity.io/glossary/vue-js>.
18. MONTERAIL. *Nuxt: the Framework for High-Performing Websites* [online]. 2025. [cit. 2026-03-06]. Dostupné z : <https://www.monterail.com/blog/nuxt-framework-for-high-performing-websites>.
19. MOZILLA. *SPA (Single-page application) - Glossary / MDN* [online]. 2025. [cit. 2026-03-06]. Dostupné z : <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
20. CURITY. *An Introduction to Single Sign-On* [online]. 2024. [cit. 2026-03-07]. Dostupné z : <https://curity.io/resources/learn/single-sign-on-introduction/>.
21. MINIORANGE. *What is Shibboleth and its Single Sign-On Architecture?* [online]. 2023. [cit. 2026-03-07]. Dostupné z : <https://www.miniorange.com/blog/what-is-shibboleth/>.

22. HAZLEWOOD, Les. *jwtk/jjwt: Java JWT: JSON Web Token for Java and Android* [online]. 2025. [cit. 2026-03-07]. Dostupné z : <https://github.com/jwtk/jjwt>.
23. PETERS, Denis. *Database IDEs in comparison* [online]. 2025. [cit. 2026-03-23]. Dostupné z : <https://blog.doubleslash.de/en/software-technologien/datenbanktechnologien/database-ides-in-comparison/>.
24. FEUERSTEIN, Steven; PRIBYL, Bill. *Oracle pl/sql Programming* [online]. O'Reilly Media, Inc., 2005 [cit. 2026-03-23].
25. VUE.JS. *Computed Properties and Watchers — Vue.js* [online]. 2022. [cit. 2026-03-14]. Dostupné z : <https://v2.vuejs.org/v2/guide/computed.html>.
26. ORACLE. *Data Types* [online]. 2025. [cit. 2026-03-16]. Dostupné z : <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Data-Types.html#GUID-A3C0D836-BADB-44E5-A5D4-265BA5968483>.
27. GEEKSFORGEES. *Manual Testing - Software Testing* [online]. 2019. [cit. 2026-03-20]. Dostupné z : <https://www.geeksforgeeks.org/software-testing/software-testing-manual-testing/>.
28. OSHEROVE, Roy. *The Art of Unit Testing: With Examples in. Net* [online]. Manning Publications Co., 2009 [cit. 2026-03-23].
29. POWELL, Phill; SMALLEY, Ian. *What is Integration Testing? | IBM* [online]. 2025. [cit. 2026-03-20]. Dostupné z : <https://www.ibm.com/think/topics/integration-testing>.

Obsah príloh

/	
└─	readme.txt.....stručný popis obsahu média
└─	src
└─	refactored_procedures..zdrojové kódy refaktorovaných procedur
└─	survey_admin_backend.....zdrojové kódy backendové časti implementace
└─	survey_admin_frontend..... zdrojové kódy frontendové časti implementace
└─	thesis.....zdrojová forma práce ve formátu L ^A T _E X
└─	Novinky_admin_rozhraenie.pdf.....PDF dokumentácia zmien
└─	atelier_survey.sql.....PL/SQL balík pre ateliérovu anketu
└─	text
└─	thesis.pdf.....text práce ve formátu PDF